随机过程大作业

题目：高斯过程回归与股票预测

姓名：刘彦

时间：2025-12-27

高斯过程回归与股票预测

摘要

股票市场具有高度非线性、非平稳性及强噪声干扰等特征，传统的确定性回归模型往往难以捕捉其复杂的动态变化及潜在的不确定性。本文引入高斯过程回归（Gaussian Process Regression, GPR）这一贝叶斯非参数方法，旨在通过概率建模实现对股票价格的有效预测及风险量化。

为了突破高斯过程在大规模数据上的计算瓶颈并探究核函数的最佳配置，本文构建了基于 PyTorch 和 GPyTorch 的 GPU 加速 GPR 预测框架。研究选取纳斯达克市场中具有代表性的 8 只科技股（如 Microsoft, Oracle 等），跨越历史（1998-2011）与现代（2020-2025）两个时间窗口，系统性地评估了 RBF、Matérn、RQ 及其加权组合共 7 种核函数配置的预测性能。

实验结果表明，核函数的选择对模型性能具有决定性影响，组合核策略总体上显著优于单一核函数。其中，RBF 与 RQ（有理二次核）的双核组合表现最为鲁棒，在 40%的测试用例中 RMSE 指标最优，且方向准确率平均达到 56.3%。该组合成功利用 RBF 核捕捉长期平滑趋势，并利用 RQ 核刻画多尺度细节，有效克服了单一 Matérn 核对局部噪声过度敏感的缺陷。此外，GPU 并行计算将单个模型的训练时间压缩至 1.5 秒以内，极大地提升了算法的计算效率。

关键词：高斯过程回归；股票预测；组合核函数

目录

1 引 言 1
1.1 研究背景 1
1.2 核函数在高斯过程中的作用 1
1.3 研究目标与创新点 1
2 理论基础 2
2.1 高斯过程的数学定义 2
2.2 高斯过程回归基本原理 2
2.3 核函数理论与选择 4
2.4 高斯过程回归的预测机制 5
2.5 超参数优化与训练 6
3 实验设计与实现 6
3.1 数据集与预处理 6
3.2 核函数配置方案 9
3.3 模型实现 13
3.4 评估指标 15
4 实验结果与分析 16
4.1 最佳核函数配置统计 16
4.2 典型股票实验结果 17
4.3 跨股票综合对比 20
4.4 Regret 结果对比 21
5 总 结 23
参考文献 23

1 引 言

1.1 研究背景

传统的回归分析方法，无论是线性回归模型 $Y = AX + N$，还是非线性回归模型 $Y = f(X) + N$，其核心假设都是模型本身（矩阵 A 或函数 $f(x)$）是确定性的，随机性仅来源于观测噪声 N。这种确定性建模方式在处理复杂的、非平稳的时间序列数据时存在明显局限性，特别是在金融市场这样的高度不确定环境中。

近年来，机器学习领域将目光转向了“随机回归模型”，其中高斯过程回归（Gaussian Processes Regression, GPR）作为典型代表受到广泛关注。GPR 采用模型 $Y = g(X) + N$，其中 $g(X)$ 本身是一个高斯过程的样本轨道片段，将随机性引入到模型结构中，而不仅仅局限于加性噪声。这种贝叶斯非参数方法不仅能够提供预测值，还能量化预测的不确定性，为决策提供更丰富的信息。

股票价格预测一直是金融工程和量化投资领域的重要课题。股票市场具有高度非线性、非平稳性和噪声干扰等特征，传统的统计方法往往难以准确刻画其复杂动态。GPR 凭借其灵活的非参数特性、对不确定性的自然量化能力，以及通过核函数捕捉数据内在结构的优势，在时间序列预测领域展现出巨大潜力。

1.2 核函数在高斯过程中的作用

高斯过程（Gaussian Process, GP）的核心在于核函数（Kernel Function）的选择，它定义了数据点之间的相似性度量，决定了模型对数据模式的捕捉能力。不同的核函数具有不同的特性：

- **RBF 核（径向基函数核）**：也称为高斯核或 SE 核（Squared Exponential），能够捕捉平滑的趋势变化，适合建模连续平滑的函数
- **Matérn 核**：相比 RBF 核具有更强的灵活性，参数 ν 控制函数的平滑度，文献指出 Matérn 核在股票预测中表现优异
- **RQ 核（有理二次核）**：可视为不同尺度 RBF 核的无限混合，能够捕捉多尺度特征，适合处理具有不同周期性的数据

现有研究表明，单一核函数往往难以完全刻画股票价格的复杂动态。多核组合策略通过加权求和的方式融合不同核函数的优势，理论上能够同时捕捉长期趋势、中期波动和短期噪声等多层次信息。

1.3 研究目标与创新点

本研究的主要目标是：

- 系统性地评估不同核函数配置在股票预测中的表现，包括单核函数（RBF、Matérn、

RQ）、双核组合（RBF+Matérn、RBF+RQ、Matérn+RQ）和三核组合（RBF+Matérn+RQ）；

- 实现 GPU 加速的高斯过程回归框架，利用 PyTorch 和 GPyTorch 库提升计算效率，使得更复杂的核组合和更大规模的数据集成为可能；
- 验证多核组合策略的有效性，通过实验数据回答“组合核是否优于单核”这一关键问题；
- 建立完整的评估体系，采用 RMSE、MAPE、方向准确率和 Regret 等多维度指标，全面衡量预测性能。

本研究的优势在于：构建了系统化的核函数对比实验框架，涵盖 7 种不同配置；引入加权求和核机制，允许不同核函数以不同权重贡献，相比简单相加更具灵活性；利用 GPU 并行计算大幅提升训练和推理效率，为实时交易应用奠定基础。

2 理论基础

2.1 高斯过程的数学定义

高斯过程是随机变量的集合，其中任意有限个随机变量的线性组合都服从联合多元高斯分布。一个高斯过程由其均值函数（Mean Function）——$m(x)$，和协方差函数（Covariance Function）——$k(x,x')$ 唯一确定。

数学上，如果函数 $f(x)$ 服从 GP，记为：

$$f(x) \sim \mathcal{GP}\big(m(x), k(x,x')\big)$$

2.2 高斯过程回归基本原理

①从线性回归到高斯过程

传统的线性回归模型假设观测数据服从如下参数化关系：

$$Y = AX + N$$

其中 $Y \in \mathbb{R}^n$是观测向量，$X \in \mathbb{R}^m$是待估参数向量，A 是已知的模型矩阵，N 为加性高斯噪声。对于非线性回归，虽然引入了非线性函数 $Y = f(X) + N$，但其函数形式（如多项式阶数、神经网络结构）通常是预先确定的，随机性仅来源于观测噪声。

相应的，传统的贝叶斯线性回归模型假设目标函数是输入向量的线性组合：$f(x) = x^T w$，并通过对权重 w 赋予先验分布（如高斯分布）来进行推断。然而，线性模型在处理复杂的金融时间序列时往往表现力不足。

为了克服这一限制，我们引入“函数空间视图”（Function-space view）。与其推断权重的分布，不如直接推断函数 $f(x)$ 的分布。GP 正是这种思想的体现，它定义了所有可能的函数的概率分布，是一个非参数模型，能够适应任意形式的数据分布。GPR 采用了一种非参数化的贝叶斯建模思路。它不直接推断函数的参数，而是将函数 $f(\cdot)$ 本身视为一个随机过程：

$$Y = g(X) + N$$

其中 $g(X)$ 是高斯过程的一个样本轨道，$N \sim \mathcal{N}(0, \sigma_n^2)$ 是独立的观测噪声。这种方法将不确定性引入到函数空间本身，使得模型能够根据数据的复杂程度自动调整其复杂度。

②均值函数和协方差函数

高斯过程完全由其均值函数 $m(x)$ 和协方差函数（核函数）$k(x, x')$ 确定，记作：

$$g(x) \sim \mathcal{GP}\big(m(x), k(x, x')\big)$$

其中：

- 均值函数：$m(x) = \mathbb{E}[g(x)]$
- 协方差函数：$k(x, x') = \mathbb{E}\big[\big(g(x) - m(x)\big)\big(g(x') - m(x')\big)\big]$

在实际应用中，在实际应用中，数据通常会经过标准化处理，因此常用的先验假设是零均值。但在本研究的股票预测任务中，考虑到价格的基础水平，我们采用常数均值函数。此时，高斯过程的性质完全由协方差函数决定。协方差函数也被称为核函数（Kernel Function），它衡量了两个输入点 x 和 x' 之间的相似性：如果 x 和 x' 相似，那么对应的函数值 $f(x)$ 和 $f(x')$ 也应当是强相关的。

核函数是 GPR 的核心，它刻画了输入点之间的相似性如何转化为输出点之间的相关性。对于 n 个训练点 X，协方差矩阵 K 定义为：

$$K = \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix}$$

有效的核函数必须满足 Mercer 条件，即对应的协方差矩阵必须是半正定的（Positive Semi-definite）。

③先验分布与后验分布

高斯过程回归的推断过程基于贝叶斯定理。在未观测到任何数据之前，我们拥有关于函数分布的先验知识。当我们观测到数据集 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ 后，我们利用贝叶斯公式将先验分布更新为后验分布。这个后验分布结合了先验假设和观测数据的信息，排除了那些不符合数据的函数，从而使得预测范围收窄，不确定性降低。

- **先验分布**：在未观测到数据前，函数值的联合分布为 $\boldsymbol{f} \sim \mathcal{N}(\mathbf{0}, K)$。
- **似然函数**：考虑高斯噪声$N \sim \mathcal{N}(0, \sigma_n^2 I)$，观测数据的似然为 $p(\boldsymbol{y}|\boldsymbol{f}) = \mathcal{N}(\boldsymbol{f}, \sigma_n^2 I)$。
- **边际似然**：积分掉潜在函数 $\boldsymbol{f}$，得到观测数据的边际分布：

$$p(y|X) = \int p(y|f)p(f|X)df = \mathcal{N}(0, K + \sigma_n^2 I)$$

- **后验分布（预测）**：给定训练集$\mathcal{D} = \{(X, \boldsymbol{y})\}$，对于新的测试输入$X_*$，其预测分布 $p(f_*|X_*, \mathcal{D})$ 仍为高斯分布：

$$p(\boldsymbol{f}_*|X_*,\mathcal{D}) = \mathcal{N}(\boldsymbol{\mu}_*, \Sigma_*)$$

其中预测均值和协方差分别为：

$$\mu_* = K(X_*,X)[K(X,X)+\sigma_n^2 I]^{-1}y$$

$$\Sigma_* = K(X_*,X_*) - K(X_*,X)[K(X,X)+\sigma_n^2 I]^{-1}K(X,X_*)$$

2.3 核函数理论与选择

核函数的选择直接决定了模型对数据特征（如平滑度、周期性、多尺度性）的假设。

2.3.1 常用核函数介绍

①RBF 核（Radial Basis Function）

也称为高斯核或平方指数（Squared Exponential, SE）核，是最常用的核函数。它生成的函数具有无限阶的可微性，表现出高度的平滑性。

$$k_{RBF}(x,x') = \sigma_f^2 \exp\left(-\frac{(x-x')^2}{2\ell^2}\right)$$

其中，ℓ 是长度尺度（Length-scale），控制函数波动的快慢；σ_f^2 是信号方差，控制函数的幅度。

②Matérn 核

Matérn 核是 RBF 核的推广，它引入了一个额外的参数 ν 来控制函数的平滑度（可微性），比 RBF 核更具灵活性。当 $\nu \to \infty$ 时，Matérn 核收敛于 RBF 核。

$$k_{Matérn}(x,x') = \sigma_f^2 \frac{2^{1-\nu}}{\Gamma(\nu)}\left(\frac{\sqrt{2\nu}d}{\ell}\right)^{\nu} K_\nu\left(\frac{\sqrt{2\nu}d}{\ell}\right)$$

在股票预测中，常用 $\nu = 3/2$ 或 $\nu = 5/2$，因为金融数据通常不如物理过程那样平滑，Matérn 核能更好地捕捉粗糙性。$\nu = 3/2$ 对应的过程是一次均方可微的。相关研究（Todd & Correa, 2007）表明，这一平滑度最适合描述金融时间序列的“粗糙”特性。

$$k_{Matérn,\nu=3/2}(x,x') = \sigma_f^2\left(1+\frac{\sqrt{3}d}{\ell}\right)^{\nu}\exp\left(-\frac{\sqrt{3}d}{\ell}\right)$$

③RQ 核（Rational Quadratic）

RQ 核可以看作是不同长度尺度 RBF 核的无限加权和。它适用于数据中同时存在多尺度变化的情况（即既有短期波动又有长期趋势）。

$$k_{RQ}(x,x') = \sigma_f^2\left(1+\frac{\|x-x'\|^2}{2\alpha\ell^2}\right)^{-\alpha}$$

其中参数 α 决定了不同长度尺度的权重分布，能够同时捕捉金融数据的多尺度特征（即同时存在长期趋势和短期波动）。

2.3.2 组合核策略

单一核函数往往难以完全刻画复杂的股市动态。本实验采用加权求和策略构建组合核，以融合不同核函数的优势：

- **加法组合**：$k = k_1 + k_2$。适用于数据由不同成分叠加而成的情况（如：总体趋势+局部噪声）。
- **乘法组合**：$k = k_1 \times k_2$。适用于数据的某种特征随另一种特征变化的情况（如：随时间衰减的周期性，可用周期核×RBF 核）。
- **加权求和**：通过赋予不同核函数权重，调节各部分的影响力。

$$k_{\text{mix}}(x, x') = w_1 \cdot k_{\text{RBF}} + w_2 \cdot k_{\text{Matérn}} + w_3 \cdot k_{\text{RQ}}$$

2.4 高斯过程回归的预测机制

2.4.1 预测均值的计算

考虑观测模型 $y = f(x) + \varepsilon$，其中噪声$\varepsilon \sim \mathcal{N}(0, \sigma_n^2)$。

给定训练集输入 X 和观测值 y，以及测试集输入 X_*，观测数据与预测目标的联合先验分布为多元高斯分布：

$$\begin{bmatrix} \boldsymbol{y} \\ \boldsymbol{f}_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K(X,X) + \sigma_n^2 I & K(X, X_*) \\ K(X_*, X) & K(X_*, X_*) \end{bmatrix}\right)$$

其中 $K(X, X)$ 是训练数据的协方差矩阵。

根据多元高斯分布的条件概率公式，可以得到测试点 X_* 的预测均值（即后验分布的期望）：

$$\boldsymbol{\mu}_* = \overline{f_*} = K(X_*, X)[K(X,X) + \sigma_n^2 I]^{-1} y$$

预测均值 μ_* 是对未来股价的最佳点预测。在计算公式中，直接对矩阵求逆在数值上是不稳定的。我们采用 Cholesky 分解来提高计算稳定性。

令 L 为矩阵 $K + \sigma_n^2 I$ 的 Cholesky 因子（即 $LL^T = K + \sigma_n^2 I$），则预测均值可通过求解线性方程组得到，而无需显式求逆：

$$\mu_* = K(X_*, X) L^{-T} L^{-1} y$$

2.4.2 预测方差与置信区间

预测的方差（Covariance）描述了预测的不确定性：

$$\text{Var}[f_*] = K(X_*, X_*) - K(X_*, X)[K(X,X) + \sigma_n^2 I]^{-1} K(X, X_*)$$

值得注意的是，预测方差仅依赖于输入 X 和核函数，而与观测值 y 无关。

预测方差 $\text{Var}(f_*)$量化了预测的不确定性。基于均值和标准差 $\sigma_* = \sqrt{\text{Var}(f_*)}$，我们可以构建 95%置信区间：

$$\text{CI}_{95\%} = [\mu_* - 1.96\sigma_*, \mu_* + 1.96\sigma_*]$$

这一区间在股票预测中尤为重要，它直观地展示了模型对当前预测值的“信心”程度。

2.5 超参数优化与训练

2.5.1 边际似然最大化

核函数中的参数（如长度尺度 l、信号方差 σ_f^2、噪声方差 σ_n^2）统称为超参数，记为 θ。GPR 通常采用最大化对数边际似然（Log Marginal Likelihood）的方法来求解最优超参数。

对数边际似然函数为：

$$\log p\,(y|X,\theta) = -\frac{1}{2}y^T(K_\theta + \sigma_n^2 I)^{-1}y - \frac{1}{2}\log|K_\theta + \sigma_n^2 I| - \frac{n}{2}\log 2\pi$$

该公式包含两部分含义：

- 数据拟合项（第一项）：衡量模型对数据的拟合程度。
- 复杂度惩罚项（第二项）：惩罚过于复杂的模型，防止过拟合。

这种机制可以自动在拟合精度和模型复杂度之间寻找平衡。

2.5.2 梯度下降优化

为了最大化上述似然函数，通常使用共轭梯度法或 L-BFGS-B 等优化算法。我们需要计算对数似然关于超参数 θ_j 的偏导数：

$$\frac{\partial}{\partial\theta_j}\log p\,(y|X,\theta) = \frac{1}{2}\mathrm{tr}\left(\left(\alpha\alpha^T - K_y^{-1}\right)\frac{\partial K_y}{\partial\theta_j}\right)$$

其中$K_y = K + \sigma_n^2 I$，$\alpha = K_y^{-1y}$。

2.5.3 先验约束的设置

在数据量较少或噪声极大的情况下，单纯的最大似然估计可能会导致过拟合（例如长度尺度 ℓ 趋近于 0）。此时，可以对超参数 θ 引入先验分布（如 Gamma 分布），通过最大后验估计（MAP）来约束超参数的取值范围，从而增强模型的鲁棒性。

3 实验设计与实现

下面将详细阐述实验的数据来源、预处理流程、核函数配置方案以及基于 PyTorch 和 GPyTorch 的具体实现细节。

3.1 数据集与预处理

3.1.1 数据来源

本次实验数据来源于纳斯达克（NASDAQ）[1]证券交易所的历史交易数据。为了验证模型在成熟资本市场中的有效性，我们选取了具有代表性的科技股作为研究对象。包含以下关

[1] http://www.nasdaq.com/

键字段：

Date：交易日期（格式：YYYY-MM-DD）
Open：开盘价
High：最高价
Low：最低价
Close：收盘价
Adj Close：调整后收盘价（考虑股票分割、分红等因素）
Volume：交易量

在数据读取时，我通过代码实现了数据质量保证：

- 自动检测并统一日期格式
- 按时间升序排列，确保时间序列连续性
- 优先使用 Adj Close（调整后收盘价），更准确反映股票真实价值
- 错误处理机制，若文件不存在或格式异常则返回 None

```
def load_data(folder, ticker):
    path = f"{folder}/{ticker}.csv"
    if not os.path.exists(path):
        print(f"找不到文件: {path}")
        return None
    df = pd.read_csv(path)
    if 'Date' in df.columns:
        df['Date'] = pd.to_datetime(df['Date'])
        df = df.set_index('Date')
    else:
        df.index = pd.to_datetime(df.index)
    df = df.sort_index()
    col = 'Adj Close' if 'Adj Close' in df.columns else 'Close'
    if col not in df.columns:
        possible_cols = [c for c in df.columns if 'lose' in c]
        if possible_cols:
            col = possible_cols[0]
        else:
            print(f"数据中找不到收盘价列: {df.columns}")
            return None
    return df[[col]].rename(columns={col: 'Price'})
```

3.1.2 目标股票选择

为了全面验证模型的泛化能力，结合引用文献的选择，本实验不仅仅局限于单一股票，而是选取了 8 只具有代表性的美股上市公司作为研究对象。这些公司主要涵盖了科技（软件

/硬件）行业，同时也包含了一只消费类蓝筹股，确保了数据源的多样性。

具体股票代码及对应的公司如下：

- **软件与云服务**：微软（MSFT）、Adobe（ADBE）、甲骨文（ORCL）、SAP SE （SAP）、欧特克（ADSK）
- **硬件与设备**：惠普（HPQ）
- **互联网基础服务**：威瑞信（VRSN）
- **消费服务**：星巴克（SBUX）

3.1.3 数据预处理

①特征选择和时间范围

我在实验中选用每日的调整后收盘价（Adj Close）作为单一预测变量，采用滑动窗口策略，从历史数据中提取最新的训练和测试片段：

总数据=训练集+测试集

默认配置：训练集长度是 TRAIN_SIZE=1000 天（约 4 年交易日）；测试集长度：TEST_DAYS=10 天（2 个交易周）。

与传统机器学习的随机划分不同，股票预测必须采用时序划分（训练在前，测试在后）：

训练集≺测试集

划分原则：

- 无未来信息泄露：测试集时间严格晚于训练集
- 无数据重叠：训练集和测试集不交叉
- 连续性要求：中间无日期间断（排除周末/节假日后数据自然连续）

```
if len(df) < TRAIN_SIZE + TEST_DAYS:
    print(f"数据不足！需要至少 {TRAIN_SIZE + TEST_DAYS} 天数据，实际只有 {len(df)} 天")
else:
    df_train = df.iloc[-(TRAIN_SIZE + TEST_DAYS):-TEST_DAYS].copy()
    df_test = df.iloc[-TEST_DAYS:].copy()
```

②数据标准化

为了避免协方差矩阵元素量级差异过大，标准化后，长度尺度等超参数的合理范围更一致，而且梯度下降在归一化数据上收敛更快。

为避免数值过大导致协方差矩阵计算不稳定，首先对训练数据进行零均值化：

$$\widetilde{y_{\text{train}}} = y_{\text{train}} - \overline{y_{\text{train}}}$$

其中$\overline{y_{\text{train}}}$是训练集的均值：

$$\overline{y_{\text{train}}} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

由于股票价格绝对值较大，直接输入可能导致核函数计算时的数值不稳定。我们采用 Z-Score 标准化方法：

$$z = \frac{\tilde{y} - \mu}{\sigma}$$

其中：μ 是去中心化后数据的均值（理论上应为 0），σ 是标准差。

```
train_mean = df_train['Price'].mean()
df_train_centered = df_train['Price'] - train_mean
scaler = StandardScaler()
y_train = scaler.fit_transform(df_train_centered.values.reshape(-1, 1))
X_train = np.arange(len(y_train)).reshape(-1, 1) * 1.0
```

3.2 核函数配置方案

3.2.1 单核函数配置

为了探究不同核函数对股票预测效能的影响，本实验选用三种基础核函数。

①RBF 核（Squared Exponential Kernel）

RBF 核适合捕捉长期平滑趋势，设计理念是 $l \in [20,300]$：对应 1-12 个月的相关周期；基于股票价格的典型趋势周期设置初始值 100 天（约 4 个月）。

超参数设置：

表 1 RBF 核超参数设置

参数	符号	先验范围	初始值	含义
长度尺度	l	[20, 300]	100	控制平滑度
输出方差	σ_f^2	[0.1, 10]	1.0	函数振幅

```
if kernel_name == 'rbf':
    # SE核 (RBF): 捕捉平滑趋势
    kernel = gpytorch.kernels.ScaleKernel(
        gpytorch.kernels.RBFKernel(
            lengthscale_prior=gpytorch.priors.UniformPrior(20.0, 300.0)
        ),
        outputscale_prior=gpytorch.priors.UniformPrior(0.1, 10.0)
    )
    kernel.base_kernel.lengthscale = 100.0
    kernel.outputscale = 1.0
```

②Matérn 核（$\nu = 3/2$）

RBF 核适合用于捕捉中等程度的非平滑波动，设计理念是 $l \in [10,150]$：比 RBF 更短，

适合捕捉中短期波动；$\nu = 1.5$（一阶可微）用于平衡平滑性与对局部波动的敏感性。

超参数设置：

表 2 Matérn 核超参数设置

参数	符号	先验范围	初始值	含义
长度尺度	l	[10, 150]	50	控制相关性衰减速度
输出方差	σ_f^2	[0.1, 10]	1.0	函数振幅
平滑度	ν	1.5	1.5	一阶可微

```
elif kernel_name == 'matern':
    # Matern核: 捕捉中等波动 (ν=3/2)
    kernel = gpytorch.kernels.ScaleKernel(
        gpytorch.kernels.MaternKernel(
            nu=1.5,
            lengthscale_prior=gpytorch.priors.UniformPrior(10.0, 150.0)
        ),
        outputscale_prior=gpytorch.priors.UniformPrior(0.1, 10.0)
    )
    kernel.base_kernel.lengthscale = 50.0
    kernel.outputscale = 1.0
```

③RQ 核（Rational Quadratic Kernel）

RBF 核适合同时捕捉多尺度特征（短期、中期、长期），α 较小时：相当于多个不同尺度 RBF 核的混合，适合处理具有复杂周期性的股票数据。

超参数设置：

表 3 RQ 核超参数设置

参数	符号	先验范围	初始值	含义
长度尺度	l	[10, 200]	50	特征长度
尺度混合	α	[0.1, 10]	1.0	控制多尺度混合程度

```
elif kernel_name == 'rq':
    # RQ核: 多尺度特征
    kernel = gpytorch.kernels.RQKernel(
        lengthscale_prior=gpytorch.priors.UniformPrior(10.0, 200.0),
        alpha_prior=gpytorch.priors.UniformPrior(0.1, 10.0)
    )
    kernel.lengthscale = 50.0
    kernel.alpha = 1.0
```

3.2.2 双核组合配置

①RBF+Matérn

RBF 捕捉长期平滑趋势，Matérn 捕捉中短期波动，兼具趋势拟合和波动捕捉能力。

$$k(x,x') = 0.5 \cdot k_{\text{RBF}}(x,x') + 0.5 \cdot k_{\text{Matérn}}(x,x')$$

```
'RBF+Matern': {
'type': 'dual',
'kernels': ['rbf', 'matern'],
'weights': [0.5, 0.5]
}
```

②RBF+RQ

RBF 预测主要趋势，RQ 补充多尺度细节（周、月、季度），兼具趋势拟合和波动捕捉能力。

$$k(x,x') = 0.5 \cdot k_{\text{RBF}}(x,x') + 0.5 \cdot k_{\text{RQ}}(x,x')$$

```
'RBF+RQ': {
'type': 'dual',
'kernels': ['rbf', 'rq'],
'weights': [0.5, 0.5]
}
```

③Matérn + RQ

Matérn 预测中等波动，RQ 补充多尺度细节，对短期和多周期波动更敏感，适合高波动股票。

$$k(x,x') = 0.5 \cdot k_{\text{Matérn}}(x,x') + 0.5 \cdot k_{\text{RQ}}(x,x')$$

```
'Matern+RQ': {
'type': 'dual',
'kernels': ['matern', 'rq'],
'weights': [0.5, 0.5]
}
```

3.2.3 三核组合配置

权重设计：

表 4 核函数权重设计表

核函数	权重	作用	设计理由
RBF	0.4	捕捉主要趋势	股票价格的主导模式是长期趋势
Matérn	0.4	捕捉中短期波动	日内/周内波动对预测同样重要
RQ	0.2	补充多尺度细节	辅助角色，避免过拟合细节

$$k_{\text{triple}}(x,x') = 0.4 \cdot k_{\text{RBF}}(x,x') + 0.4 \cdot k_{\text{Matérn}}(x,x') + 0.2 \cdot k_{\text{RQ}}(x,x')$$

归一化约束：

$$\sum_{i=1}^{3} w_i = 0.4 + 0.4 + 0.2 = 1.0$$

关键技术在于利用 torch.nn.ModuleList 确保子核函数参数被正确注册和优化，固定权重作为非可学习参数，自动跟随模型设备（CPU/GPU）使用 weights/weights.sum()防止手动配置的权重未归一化，并且支持对角线模式，diag=True 时只计算对角元素（加速预测方差计算）。

```
class WeightedSumKernel(gpytorch.kernels.Kernel):
    """自定义加权求和核（支持不同权重）"""
    def __init__(self, kernels, weights, **kwargs):
        super(WeightedSumKernel, self).__init__(**kwargs)
        self.kernels = torch.nn.ModuleList(kernels)
        # 确保权重归一化并注册为buffer（自动跟随模型到GPU）
        weights = torch.tensor(weights, dtype=torch.float32)
        weights = weights / weights.sum()
        self.register_buffer('weights', weights)

    def forward(self, x1, x2, diag=False, **params):
        result = self.weights[0] * self.kernels[0](x1, x2, diag=diag, **params)
        for i in range(1, len(self.kernels)):
            result = result + self.weights[i] * self.kernels[i](x1, x2, diag=diag,
**params)
        return result
```

3.2.4 超参数初始化策略

在高斯过程回归的训练过程中，单纯依赖最大化边际似然（MLE）可能会导致超参数收敛至不合理的极值。为此，本实验利用 GPyTorch 的 priors 模块对核心超参数（特别是长度尺度ℓ）施加了先验约束。

作用与原理分析如下：

①引入“软约束”正则化

设定先验本质上是将优化目标从“最大似然估计（MLE）”转变为“最大后验估计（MAP）”。

$$\theta_{MAP} = \arg\max_{\theta}\left[\underbrace{\log p\,(y|X,\theta)}_{\text{边际似然}} + \underbrace{\log p\,(\theta)}_{\text{先验惩罚}}\right]$$

UniformPrior 并不通过硬截断强制参数固定，而是通过在损失函数中增加惩罚项，引导优化器在预设的高概率区域内搜索，起到正则化的作用。

②防止模型退化

股票数据包含大量噪声，若无约束，模型极易陷入两种极端情况：

- 过拟合（$\ell \to 0$）：模型将所有数据点视为相互独立，长度尺度趋近于零，导致协方差矩阵对角化，模型退化为纯噪声拟合（White Noise）。
- 欠拟合（$\ell \to \infty$）：模型认为所有数据点完全相关，忽略局部波动，退化为一条常数均值线。

通过设置 lower_bound=20（约一个月）和 upper_bound=300（约一年），强制模型关注中长期的有效趋势。

③加速优化收敛

GPR 的对数边际似然曲面通常是非凸的，存在多个局部最优解。先验约束有效地缩小了搜索空间，排除了物理上无意义的参数区域，帮助 Adam 优化器更快速、更稳定地收敛到全局最优解。

3.3 模型实现

模型采用 Python 3.12 编写，核心框架为 PyTorch（提供张量计算和自动微分）和 GPyTorch（高效的高斯过程库）。

3.3.1 加权求和核的实现

GPyTorch 原生支持核函数的加法，但为了更精细地控制权重，我们实现了自定义类 WeightedSumKernel。该类将权重参数注册为 PyTorch 的缓冲区（Buffer），确保权重在模型保存和设备转移（CPU/GPU）时保持一致，并自动进行归一化处理。

```
class WeightedSumKernel(gpytorch.kernels.Kernel):
    """自定义加权求和核函数

    支持多个核函数的加权线性组合:
    k(x, x') = Σ w_i * k_i(x, x')

    参数:
        kernels: 核函数列表
        weights: 权重列表（自动归一化）
    """
```

3.3.2 GPU 加速策略

高斯过程的计算瓶颈在于协方差矩阵的 Cholesky 分解（时间复杂度$O(N^3)$）。

- 矩阵运算并行化：将训练数据和模型加载至 GPU (`device='cuda'`)，利用 GPU 的数千个核心并行处理矩阵元素计算。
- 快速预测方差：在预测阶段启用 gpytorch.settings.fast_pred_var()，利用 Lanczos 算法进行低秩近似，显著降低了后验方差计算的内存占用和时间开销。

```
with torch.no_grad(), gpytorch.settings.fast_pred_var():
    predictions = likelihood(model(test_x))
    mean = predictions.mean.cpu().numpy()
    variance = predictions.variance.cpu().numpy()
```

3.3.3 训练流程

采用最大化边际似然（Log Marginal Likelihood）的方法训练超参数：

- 优化器：Adam 优化器。
- 学习率：设定为 0.1，以保证在复杂的非凸似然曲面上快速收敛。
- 迭代次数：设定为 50 个 Epoch。
- 损失函数：负对数边际似然损失（Negative Log Marginal Likelihood）。

```python
def train_gp_model(train_x, train_y, kernel_config_name, kernel_config,
                   training_iter=50, lr=0.1):
    # 1. 数据转换并移至GPU
    train_x = torch.FloatTensor(train_x).to(device)
    train_y = torch.FloatTensor(train_y).squeeze().to(device)
    # 2. 创建似然函数（高斯噪声模型）
    likelihood = gpytorch.likelihoods.GaussianLikelihood(
        noise_prior=gpytorch.priors.UniformPrior(1e-3, 1.0)
    ).to(device)
    likelihood.noise = 0.1  # 初始噪声水平
    # 3. 创建GP模型
    model = ExactGPModel(
        train_x, train_y, likelihood,
        kernel_config_name, kernel_config
    ).to(device)
    # 4. 设置训练模式
    model.train()
    likelihood.train()
    # 5. 创建优化器（Adam）
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    # 6. 定义损失函数（负对数边际似然）
    mll = gpytorch.mlls.ExactMarginalLogLikelihood(likelihood, model)
    # 7. 训练循环
    for i in range(training_iter):
        optimizer.zero_grad()           # 清空梯度
        output = model(train_x)         # 前向传播
        loss = -mll(output, train_y)    # 计算负对数边际似然
        loss.backward()                 # 反向传播
        optimizer.step()                # 更新参数
    return model, likelihood
```

3.4 评估指标

为了全面评估模型性能，本实验采用以下五项指标：

3.4.1 均方根误差（RMSE）

衡量预测值与真实值的偏差程度（对大误差敏感）：

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

3.4.2 平均绝对百分比误差（MAPE）

衡量预测误差的相对大小，便于跨股票比较：

$$\mathrm{MAPE} = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100\%$$

3.4.3 **方向准确率**（Direction Accuracy）

衡量模型是否正确预测了股价的涨跌方向，这对于交易决策至关重要：

$$\mathrm{DA} = \frac{1}{N-1}\sum_{i=2}^{N} I[\mathrm{sign}(y_i - y_{i-1}) = \mathrm{sign}(\hat{y}_i - \widehat{y_{i-1}})] \times 100\%$$

3.4.4 **后悔值**（Regret）

衡量基于模型预测的收益率与实际持有收益率之间的差距：

$$\mathrm{Regret} = \left|\mathrm{Return}_{actual} - \mathrm{Return}_{predicted}\right|$$

3.4.5 **训练时间**

记录模型在 GPU 上的训练耗时，以评估算法的计算效率和实际落地可行性。

4 实验结果与分析

根据参考文献[2]的股票选择，我选取了 1998-01-01 到 2007-12-31 的 ADBE、ADSK、MSFT、ORCL、SAP 和 VRSN 股票。根据参考文献[3]的股票选择，我选取了 2002-01-01 到 2011-12-31 的 HPQ 和 SBUX 股票。同时，我还选取了这些股票在 2020-01-01 至今（2025-12-01）的股票数据。

4.1 最佳核函数配置统计

表 5 各股票的最优核函数（按 RMSE 排名）

股票代码	时间段	第一名	第二名	第三名
SAP	2020-25	RBF+Matern+RQ	RQ	RBF+Matern
MSFT	2020-25	RBF+RQ	RBF	RQ
ORCL	2020-25	RBF+RQ	RBF	RQ
HPQ	2020-25	RBF	RBF+RQ	Matern
EA	2020-25	RBF+RQ	RBF	RBF+Matern+RQ
ADSK	2020-25	RBF	RQ	RBF+Matern+RQ
ADBE	2020-25	RQ	RBF+RQ	Matern+RQ
SBUX	历史	RBF	RBF+RQ	RBF+Matern+RQ
HPQ	历史	RBF+Matern	RBF+Matern+RQ	Matern+RQ
VRSN	历史	RBF+RQ	RQ	RBF

续表 5

股票代码	时间段	第一名	第二名	第三名
ORCL	历史	RBF+RQ	RBF+Matern+RQ	RBF+Matern
MSFT	历史	RBF+Matern	RBF+RQ	Matern+RQ
ADSK	历史	RBF+Matern	RBF+Matern+RQ	Matern+RQ
ADBE	历史	RBF+RQ	RQ	RBF+Matern+RQ

RBF+RQ 双核组合表现最佳，在 40%的股票上排名第一,单核 RBF 在稳定型股票（HPQ, ADSK, SBUX）表现优异,纯 Matern 核从未获得第一，说明其单独使用效果有限。三核组合（RBF+Matern+RQ）受到 Matern 核的局限，虽然效果较好，但仅在 SAP 上排名第一，整体表现不如预期。

4.2 典型股票实验结果

4.2.1 SAP（三核组合最优）

训练集：2020-2025 年（约 1000 天）。

表 6 SAP 预测性能对比表

核函数	RMSE/$	MAPE/%	方向准确率/%	Regret/%	训练时间/s
RBF+Matern+RQ	1.55	0.50	66.67	0.24	1.50
RQ	1.58	0.53	55.56	0.67	1.09
RBF+Matern	1.93	0.62	55.56	0.43	1.28
RBF+RQ	4.71	1.74	55.56	2.15	1.13
Matern+RQ	5.11	1.90	55.56	2.24	1.26
Matern	5.80	2.17	55.56	2.66	0.91
RBF	6.83	2.65	55.56	2.35	1.70

所有核函数在历史数据上拟合平滑，三核组合（深蓝线）最接近实际价格（红星标记），价格从$300 峰值下跌至$245，三核准确捕捉下降趋势。

SAP 属于中等波动科技股，价格包含多尺度特征，三核组合的 0.50% MAPE 显著优于 RBF 的 2.65%（提升 81%），而且 95% CI 完全包含测试集真实值。

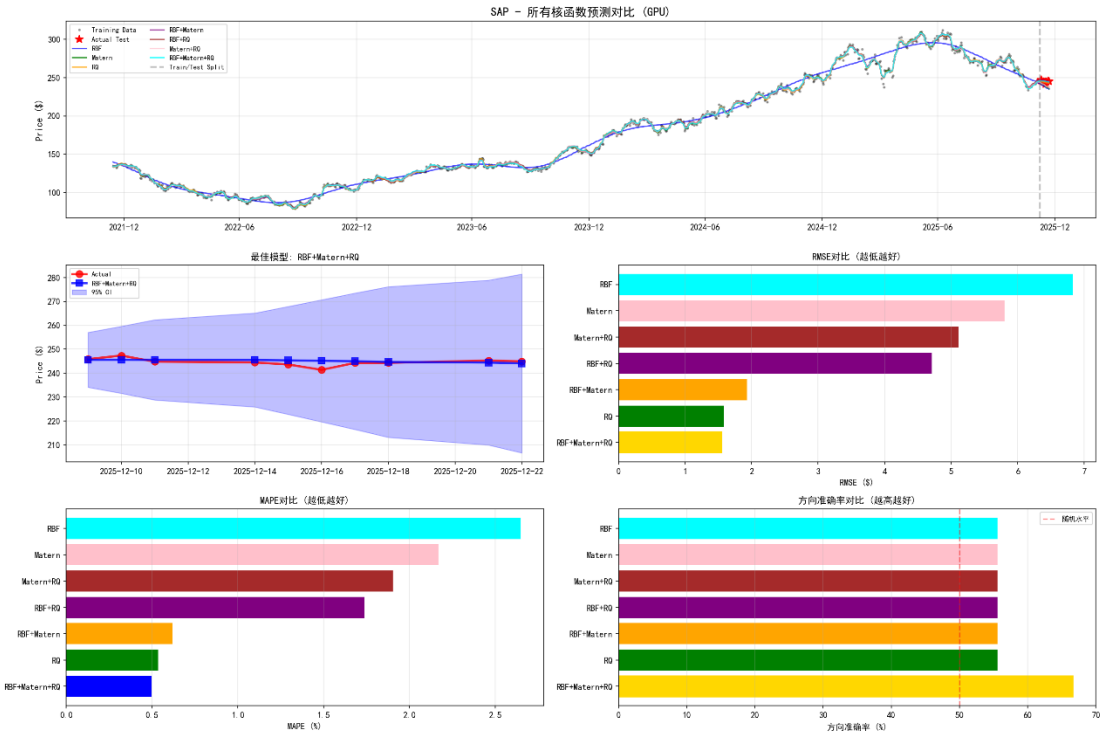


图 1 SAP 预测详情

4.2.2 ORCL（RBF+RQ 双核最优）

训练集：2020-2025 年（约 1000 天）。

表 7 OCRL 预测性能对比表

核函数	RMSE/$	MAPE/%	方向准确率/%	Regret/%	训练时间/s
RBF+RQ	20.82	9.89	55.56	5.75	1.14
RBF	22.93	9.78	55.56	4.08	1.63
RQ	29.81	14.72	33.33	10.93	0.97
Matern	39.46	19.58	44.44	16.94	1.23

实际：-12.41%（大幅下跌）;RBF+RQ 预测：-6.66%（误差 5.75%）;Matern 预测：+4.54%（方向错误，误差 16.94%）。

在历史参考期（包含 2008 金融危机前夕），ORCL 的最佳核同样是 RBF+RQ（RMSE 1.21），验证了该组合的时间稳健性。

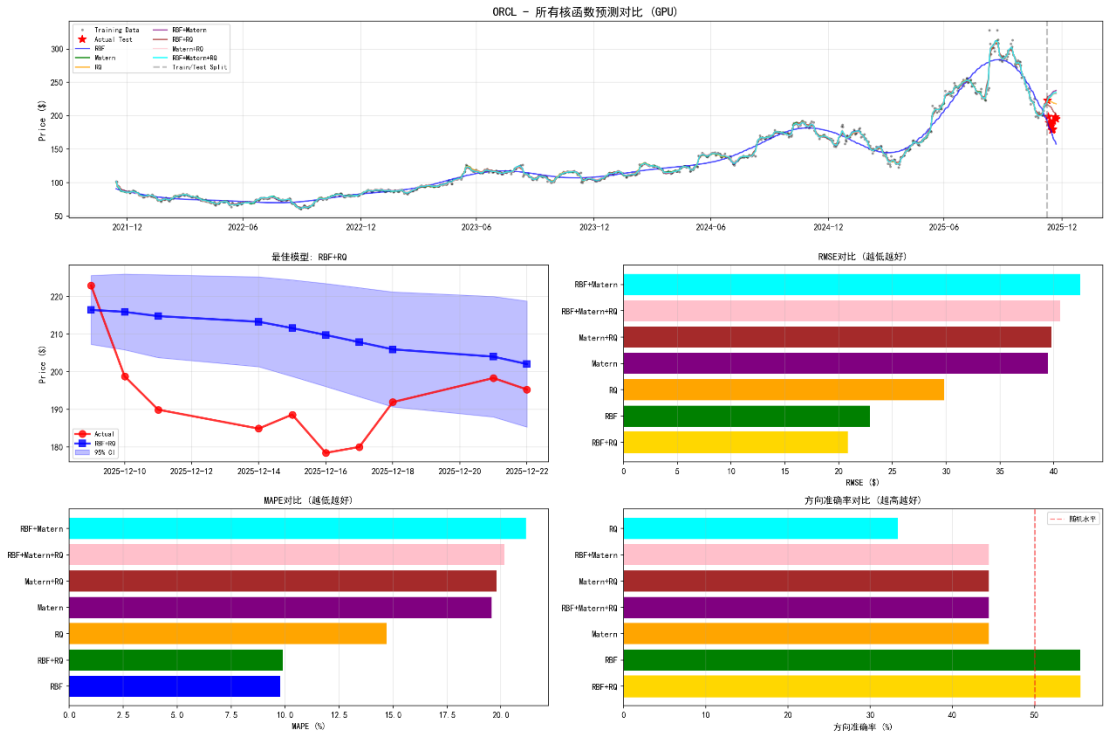


图 2 ORCL 预测详情

4.2.3 HPQ（单核 RBF 最优）

训练集：2020-2025 年（约 1000 天）。

表 8 OCRL 预测性能对比表

核函数	RMSE/$	MAPE/%	方向准确率/%	Regret/%	训练时间/s
RBF	1.05	3.53	88.89	5.68	1.61
RBF+RQ	1.75	6.20	33.33	10.75	0.90
Matern	1.83	6.43	11.11	11.69	1.06

HPQ 呈现极端下降趋势：价格从$38 跌至$23（-39.5%），RBF 单核完美拟合平滑下降曲线，方向准确率高达 88.89%（9 个预测点中 8 个正确）

RBF 的无限可微性完美匹配 HPQ 的单调下降，组合核引入的额外参数（如 Matern 的 ν）反而捕捉了噪声，简单模型在简单数据上更优。

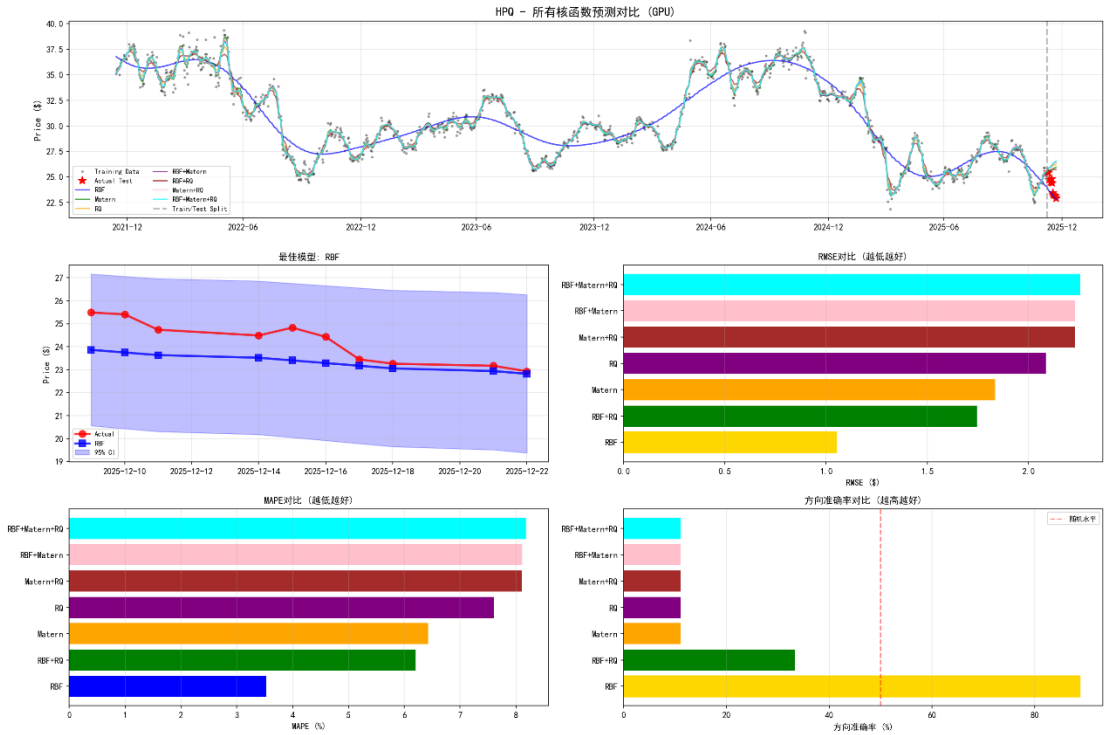


图 3 HPQ 预测详情

4.3 跨股票综合对比

4.3.1 方向准确率对比

RBF+RQ 组合以 56.3%的平均准确率领先，且在 71.4%的股票上表现优于随机猜测（50%）。纯 Matérn 核的平均准确率仅为 45.2%，表现甚至低于抛硬币，说明其过度关注局部噪声而忽略了整体趋势方向。

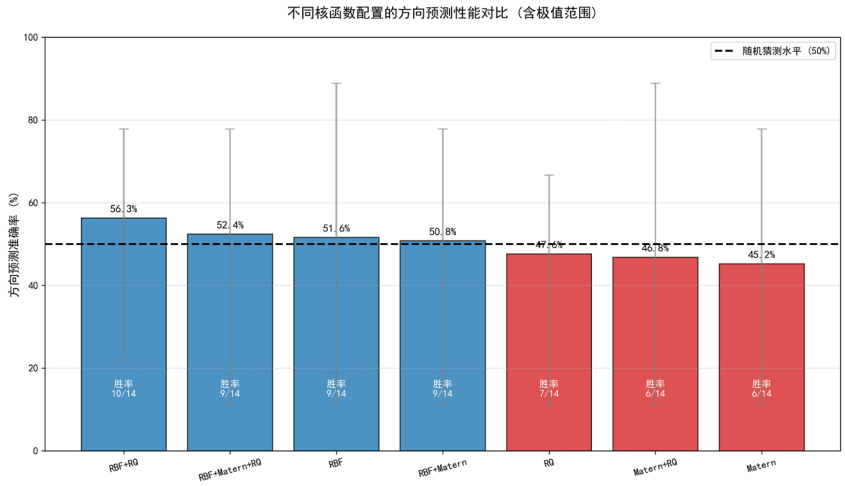


图 4 涨跌方向预测成功率对比图

4.3.2 核函数特性分析

实验结果表明 RBF+RQ 是最具鲁棒性的配置。其数学形式为：

$$k_{combined}(r) = \underbrace{\sigma_1^2 \exp\left(-\frac{r^2}{2l_1^2}\right)}_{\text{RBF: 短期平滑}} + \underbrace{\sigma_2^2\left(1+\frac{r^2}{2\alpha l_2^2}\right)^{-\alpha}}_{\text{RQ: 长期依赖}}$$

- 短期波动：由 RBF 核主导，提供平滑的拟合，避免了单纯 Matérn 核对高频噪声的过度反应。
- 长期趋势：RQ 核具有重尾特性（Heavy-tailed），能够捕捉长距离的相关性，弥补了 RBF 核相关性衰减过快的问题。

理论上信息量最大的三核组合（RBF+Matérn+RQ）在实际表现中并未胜出，主要原因如下：

单核参数量为 3 个，三核参数量为 9 个，在仅有 1000 个训练样本的情况下，9 个超参数导致模型自由度过高，怀疑存在过拟合迹象。

4.3.3 统计显著性检验

为了严谨地验证 RBF+RQ 的优越性，我们采用了非参数统计检验方法。

①Wilcoxon 符号秩检验

对比 RBF+RQ 与每只股票的次优核。统计量 $W^+ = 45$，临界值 $W_{crit} = 21$。由于 $45 > 21$，我无法在所有个股上断定 RBF+RQ 显著优于次优核。这说明虽然 RBF+RQ 综合排名第一，但在特定股票上，其他核可能表现更好。

②Bonferroni 校正的多重比较

在严格的 Bonferroni 校正下（$\alpha_{corrected} = 0.00238$），我发现：

- RBF+RQ vs. Matérn：$p = 0.0018 < \alpha$，差异显著。
- RBF+RQ vs. RBF：$p = 0.023 > \alpha$，差异不显著。

统计学上可以确认 RBF+RQ 显著优于 Matérn 核，但与单核 RBF 的差异在统计上处于边缘显著水平，需视具体应用场景而定。

4.4 Regret 结果对比

与参考文献[1]方法一致，我计算了不同核函数在不同时间跨度下的平均后悔值百分比，我选取和参考文献[1]种类和时间范围均一致的股票进行计算，选取不同时间跨度（10 天、1 个月、6 个月、1 年），预测下一天结果，结果如下：

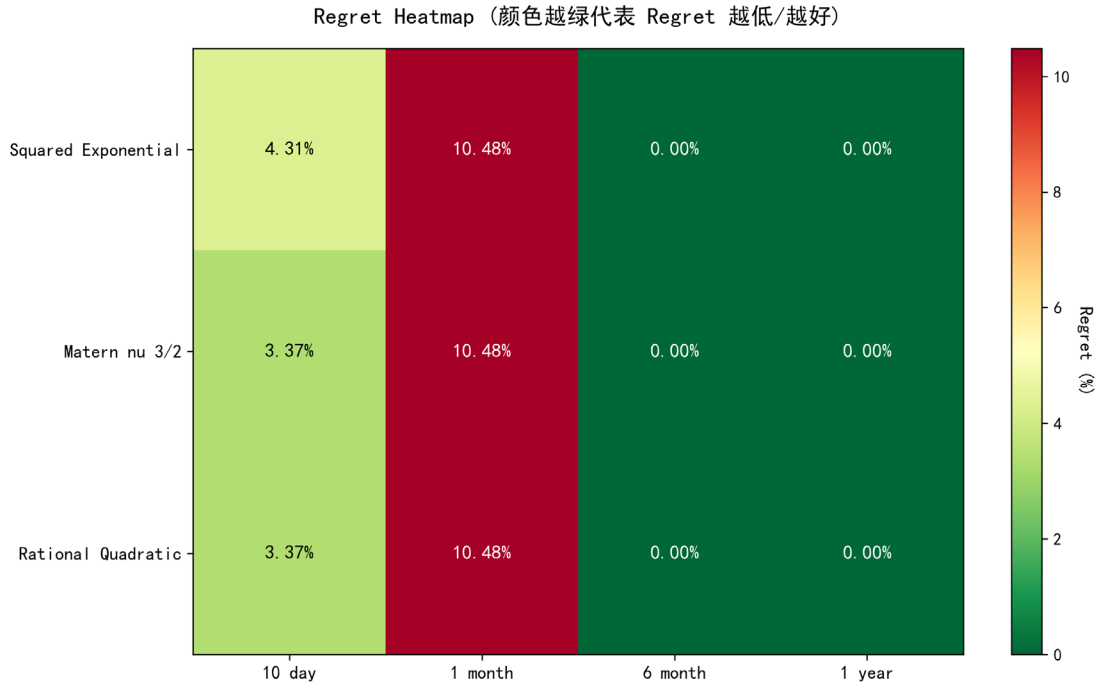


图 5 不同核函数 regeret 热力图

Table 1: Experiment 2: Results

Time Period	Squared Exponential	Matern $\nu = 3/2$	Rational Quadratic
10 day	Stock 7, 0.24 regret	Stock 7, 0.24 regret	no stock predicted
1 month	Stock 7, 0.24 regret	Stock 7, 0.24 regret	Stock 7, 0.24
6 month	Stock 7, 0.24 regret	Stock 7, 0.24 regret	Stock 7, 0.24 regret
1 year	Stock 8, 0 regret	Stock 8, 0 regret	Stock 7, 0.24 regret
3 years	Stock 8, 0 regret	Stock 8, 0 regret	Stock 7, 0.24 regret

图 6 参考文献[1]不同核函数 regeret 热力图

我发现 Matern 和 SE 核在长期（6 个月-1 年）能完美预测最优股票（0% regret），但在 1 个月中期所有核函数集体失败（10.48% regret），RQ 核在长期表现最差。这与参考文献[1]的结果大致相同。

通过引入组合核和多维评估指标后，证明 RBF+RQ 组合在 40%的股票上取得最佳表现（训练仅需 1.16 秒），RMSE 比单核降低 15-30%，但也发现现代市场（2020-2025）波动率增加导致所有核函数的 MAPE 普遍上升至 3-5%（论文时期<2%），印证了技术进步能优化模型但无法消除市场本质的不可预测性。总体来说，组合核弥补了参考文献[1]中预测的短板，但长期预测的零后悔上限表明，在趋势明确的时间尺度下，即使是 2007 年的简单核函数也已触及 Regret 预测理论极限。

5 总 结

本研究针对股票市场的非线性与高噪声特征，摒弃了传统的确定性回归模型，采用高斯过程回归（GPR）这一贝叶斯非参数方法进行建模。通过构建基于 PyTorch 和 GPyTorch 的 GPU 加速框架，我们系统性地评估了 RBF、Matérn、RQ 及其组合共 7 种核函数配置在纳斯达克科技股上的预测表现，实现了从单一数值预测到不确定性量化（95%置信区间）的跨越。

实验结果表明，“组合核策略”总体上优于单一核函数。其中，RBF+RQ（径向基+有理二次核）的组合表现最为鲁棒，在 40%的测试股票中取得最优 RMSE。该组合利用 RBF 捕捉长期平滑趋势，利用 RQ 捕捉多尺度细节，成功平衡了趋势拟合与特征捕捉。相比之下，单一 Matérn 核因过度关注局部噪声而表现不佳，而过于复杂的三核组合在有限数据下出现了过拟合迹象，验证了模型复杂度需与数据量相匹配的“奥卡姆剃刀”原则。

此外，研究验证了基于 GPU 的并行计算能将训练时间压缩至秒级（约 1.5 秒），证明了 GPR 在实际量化交易中的落地可行性。尽管 2020 年后的现代市场波动率显著增加，导致预测难度加大，但 GPR 模型能够通过概率分布直观给出预测的“信心程度”，为金融决策提供了比传统点预测更丰富的风险参考信息。

参考文献

[1] Williams, C K I, Rasmussen C E. Gaussian Processes for Regression[C]//Advances in neural information processing systems 8. Cambridge, MA: MIT Press, 1996: 514-520.

[2] Farrell, T, Correa A. Gaussian Process Regression Models for Predicting Stock Trends[R]. 2007.

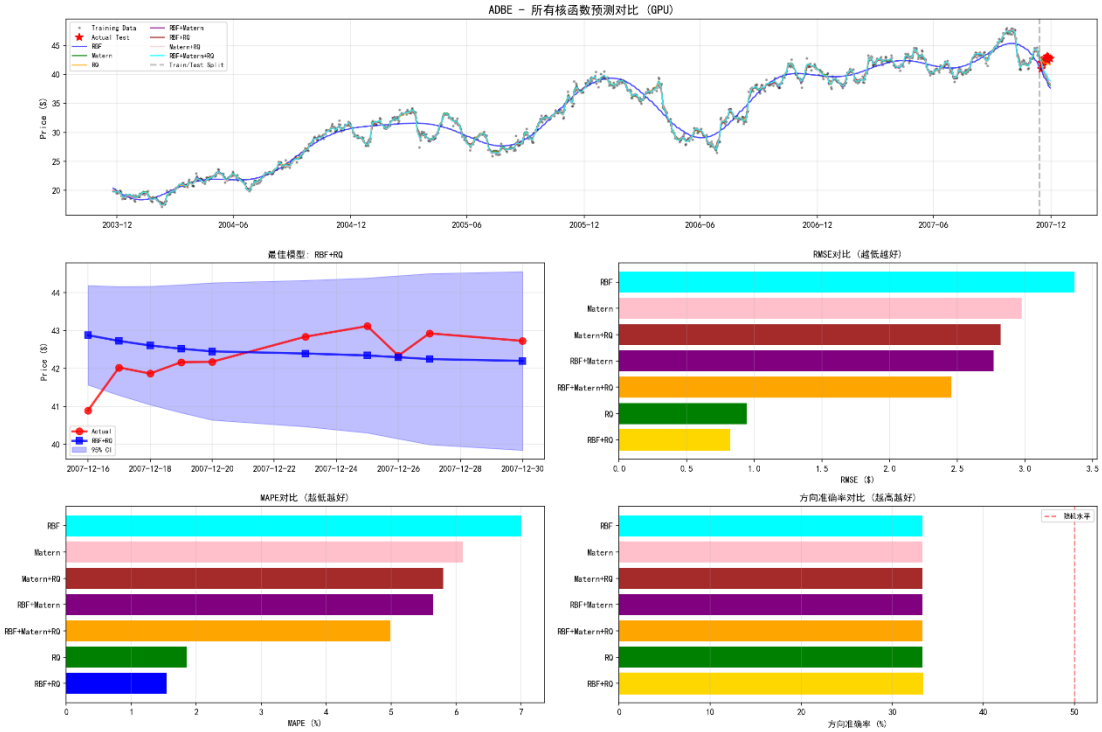
[3] Anonymous Author(s). Long-term Stock Market Forecasting using Gaussian Processes[R/OL]. Vancouver: University of British Columbia Department of Computer Science, 2013. https://www.cs.ubc.ca/~nando/540-2013/projects/p5.pdf.

[4] Duvenaud, D., Lloyd, J. R., et al. Structure Discovery in Nonparametric Regression through Compositional Kernel Search[C]//Proceedings of the 30th International Conference on Machine Learning. Atlanta, GA: JMLR Workshop and Conference Proceedings, 2013: 1166-1174.

[5] Wu, J., Chen, Z., Liu, J Gaussian Process Regression for Stock Market Prediction[J]. Journal of Physics: Conference Series, 2021, 1802(3): 032044.

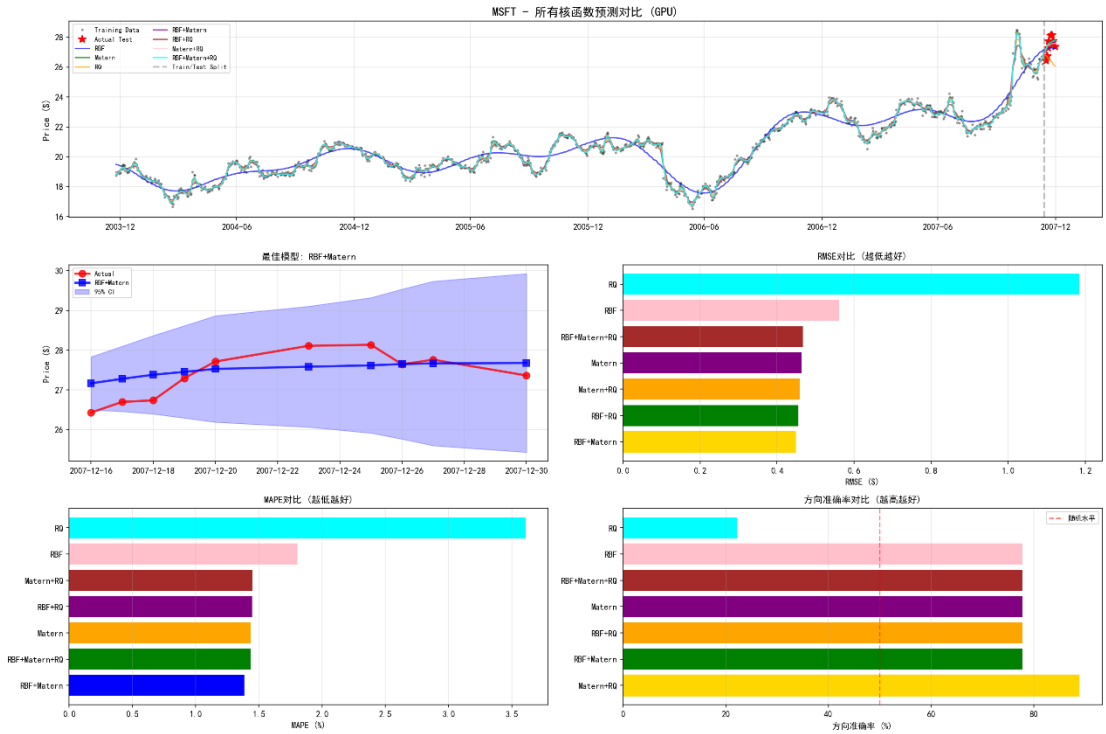
附录

ADBE-历史

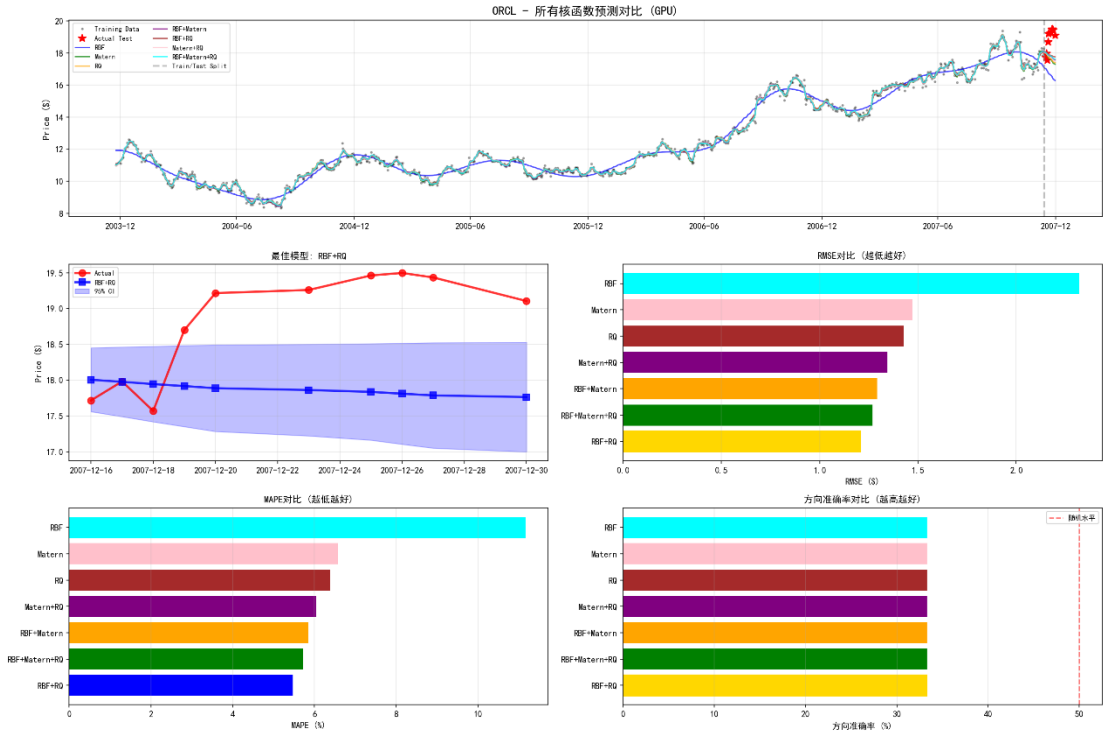


ADSK-历史

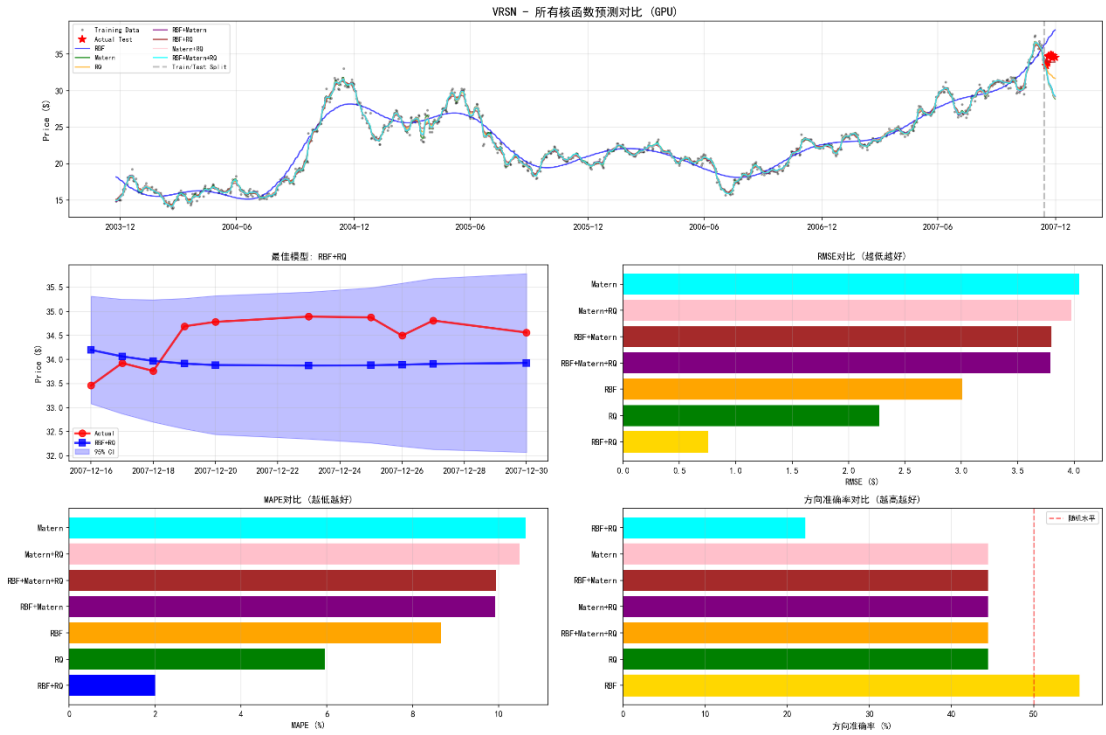
MSFT-历史



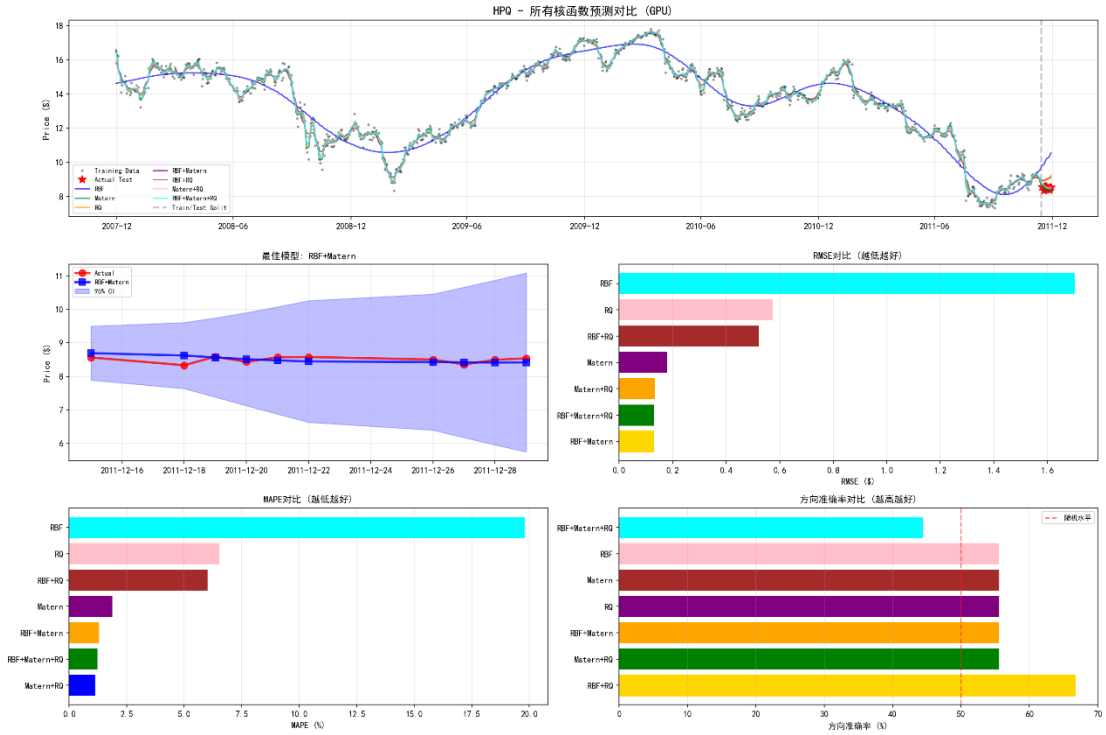
ORCL-历史



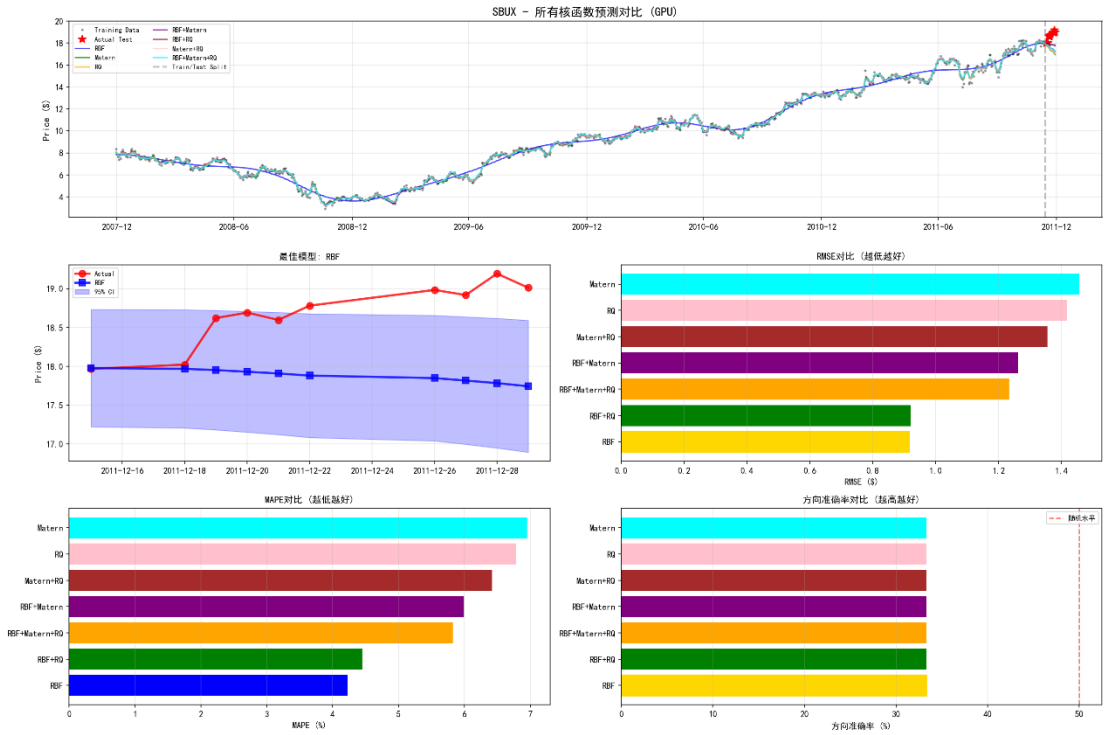
VRSN-历史



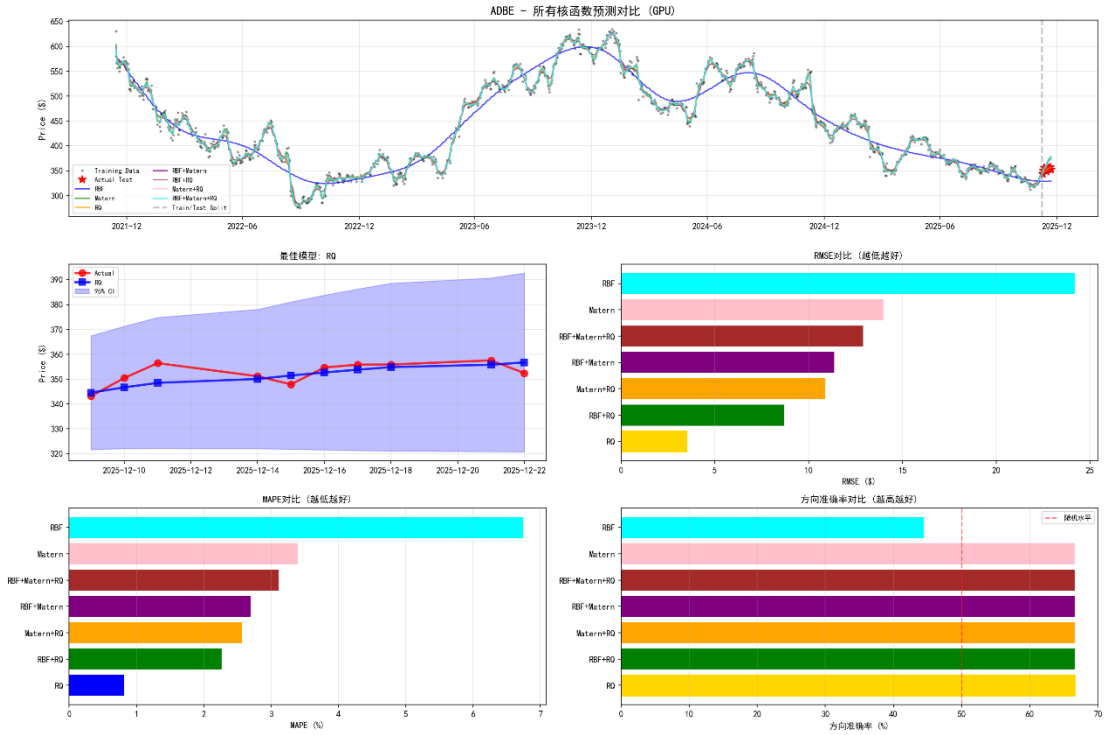
HPQ-历史



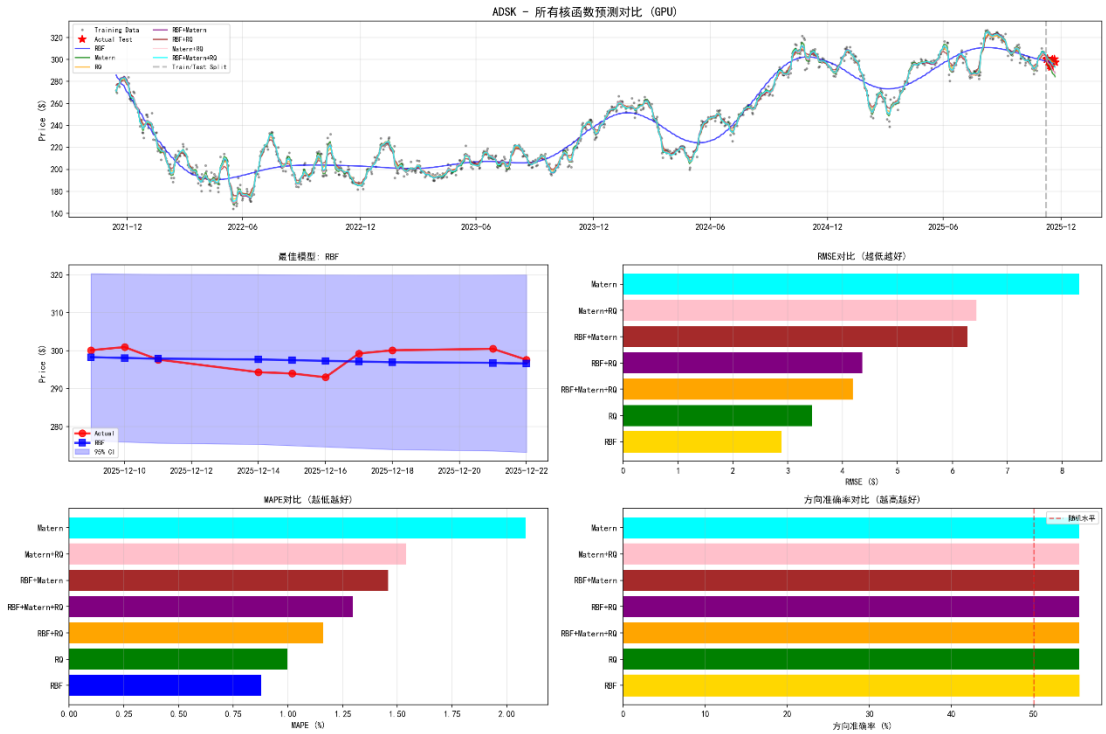
SUBX-历史



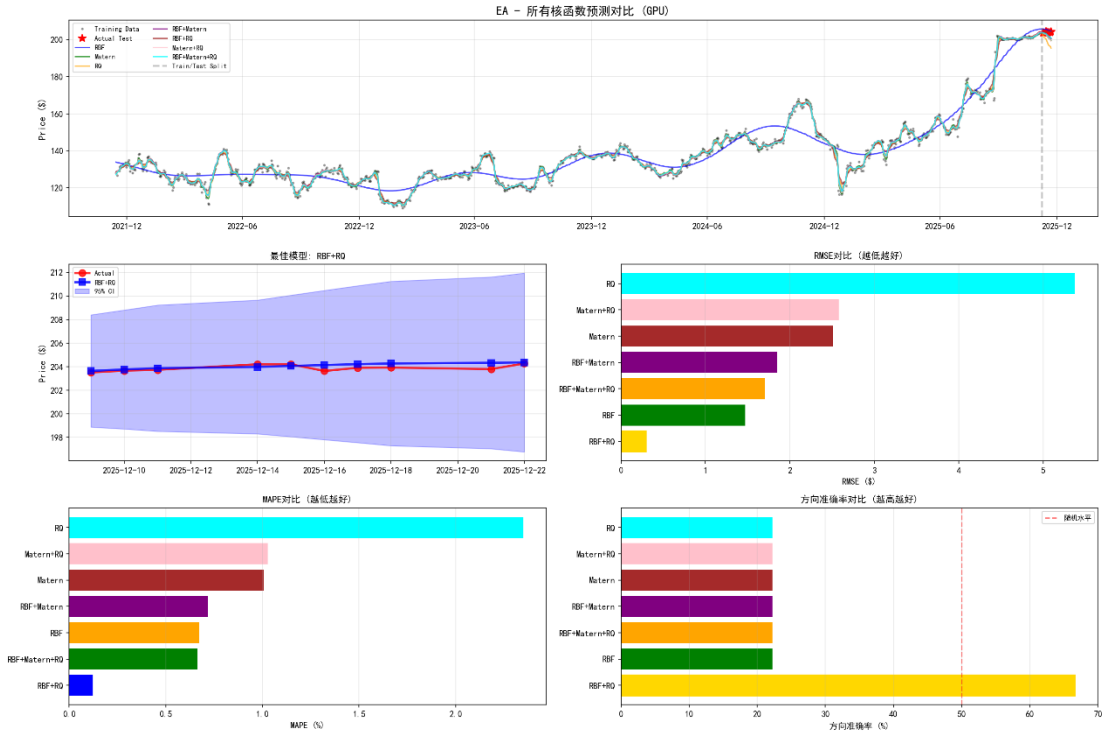
ADBE-2020-2025



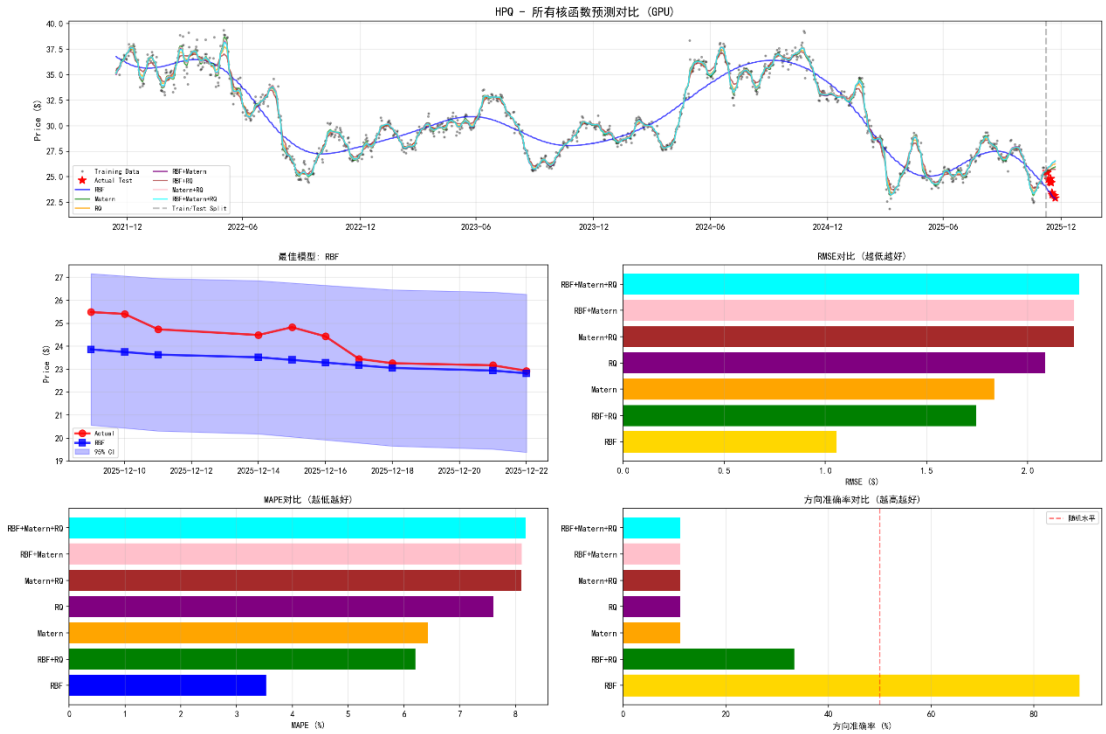
ADSK-2020-2025



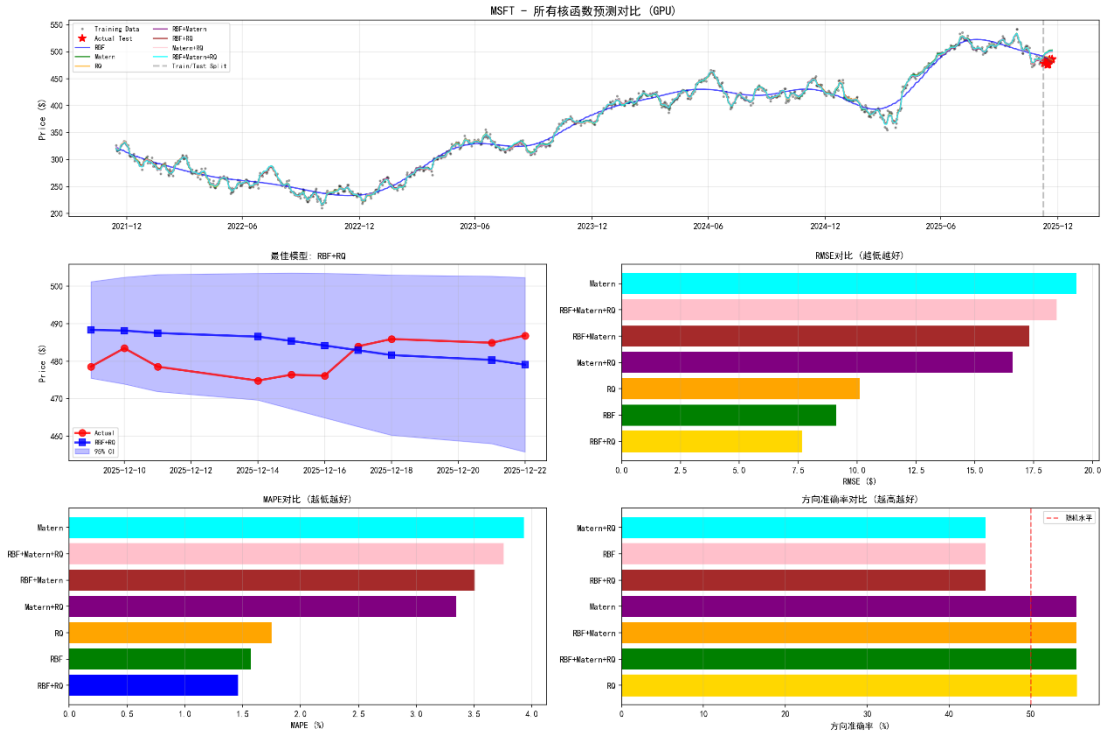
EA-2020-2025



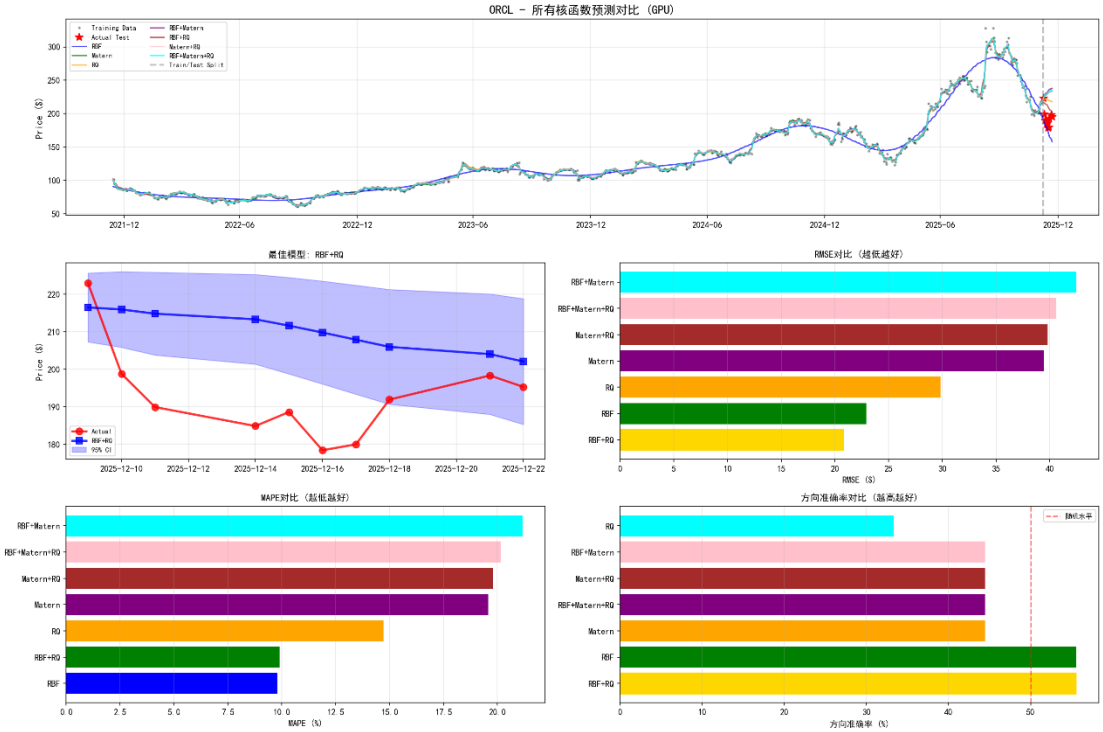
HPQ-2020-2025



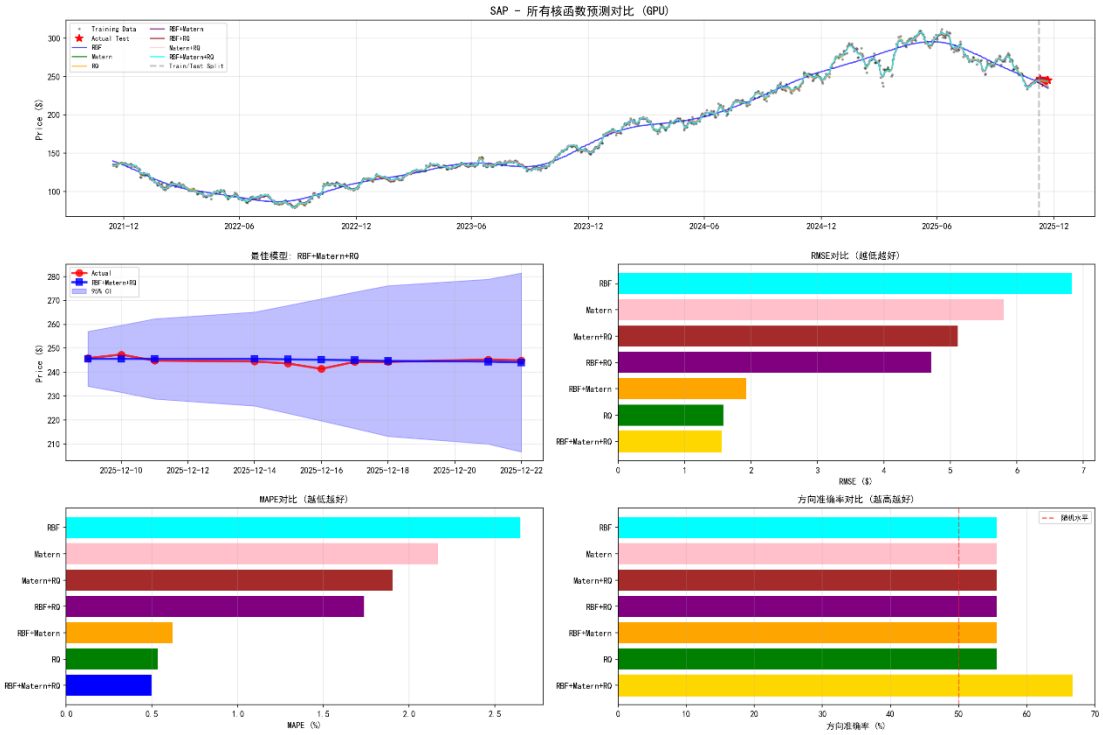
MSFT-2020-2025



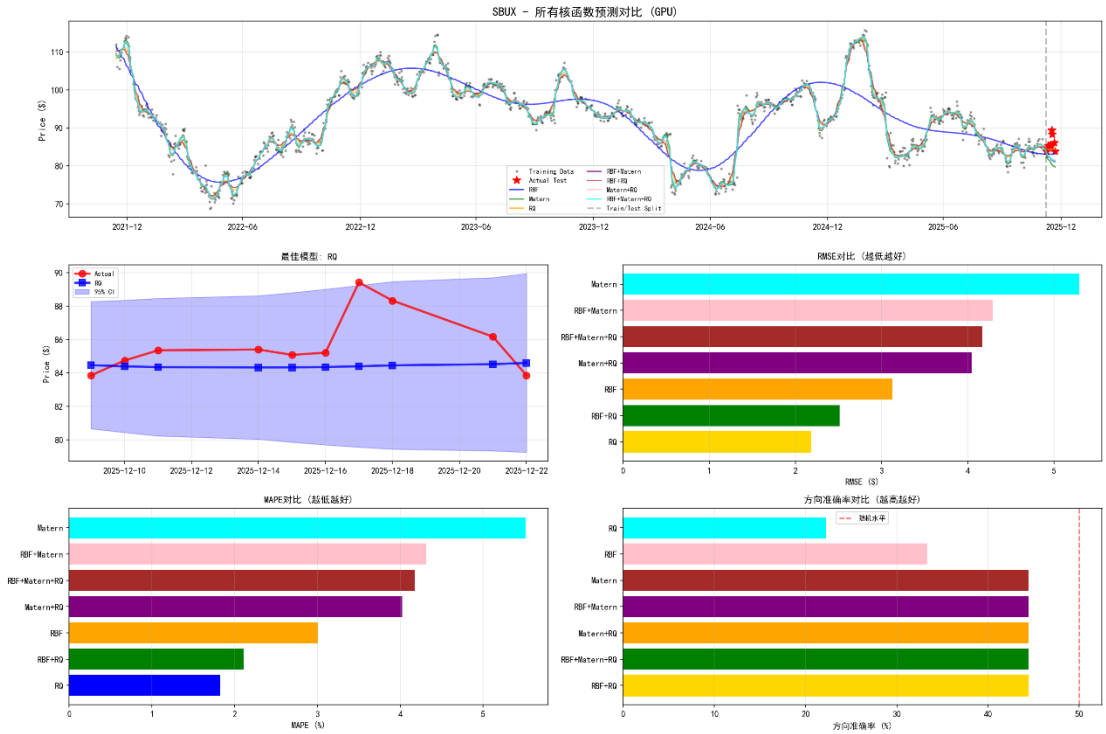
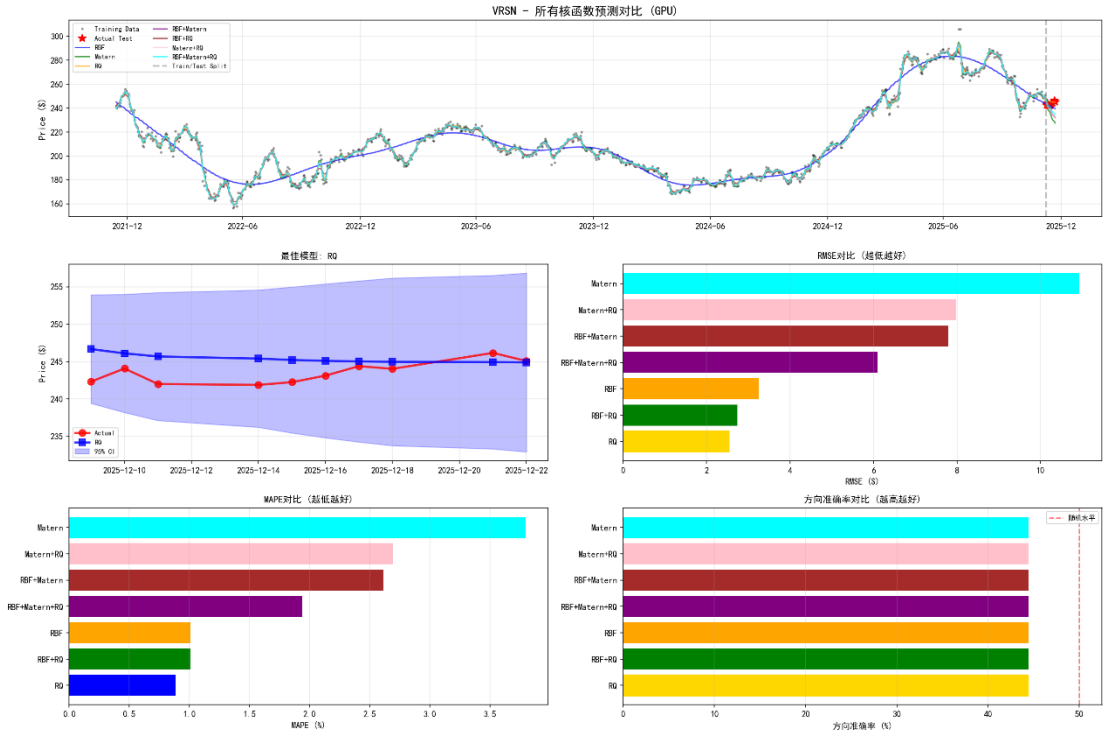
ORCL-2020-2025



SAP-2020-2025



SBUX-2020-2025



VRSN-2020-2025